

Enhancing Retrieval Quality in RAG-Based Defensive Security Systems through Query Reformulation

Subhodeep Sarkar – 253000118 (M.Tech CSE)

Nitish Kumar Bind – 253000112 (M.Tech CSE)

International Institute of Information Technology, Naya Raipur, CG

Abstract—This paper investigates whether large-language-model-based query reformulation can improve retrieval quality in defensive security retrieval-augmented generation systems. The project targets analyst-facing cybersecurity scenarios where users often ask vague questions such as “something weird in my network traffic” while the underlying corpus contains structured terminology from MITRE ATT&CK, NVD CVE records, and CISA advisories. We build a baseline RAG pipeline and four reformulation pipelines: query expansion, hypothetical document embedding (HyDE), step-back prompting, and sub-query decomposition. The system is implemented locally with Ollama using Llama 3.2 for generation and reformulation, nomic-embed-text for embeddings, and ChromaDB as the persistent vector store. Evaluation over 15 security queries shows that reformulation is not uniformly helpful: HyDE achieves the best mean retrieval performance overall with Recall@5 of 0.383 and MRR of 0.639, while query expansion and step-back prompting can degrade performance when reformulations drift away from the original intent. The findings show that reformulation should be applied selectively in practical defensive security systems, especially when analyst queries are vague, incomplete, or poorly aligned with corpus terminology.

Keywords—retrieval-augmented generation, cybersecurity, query reformulation, defensive security, MITRE ATT&CK, vector search, Ollama

I. INTRODUCTION

Modern defensive security operations increasingly depend on fast access to large bodies of structured and semi-structured knowledge, including threat-intelligence reports, MITRE ATT&CK techniques, vulnerability records, and remediation advisories. Retrieval-Augmented Generation (RAG) offers a practical way to combine semantic retrieval with large language model reasoning so that an analyst can ask a natural-language question and receive an answer grounded in supporting evidence.

The central problem addressed in this project is query-document mismatch. Security analysts and students often phrase questions in vague, operational, or symptom-based language, whereas the indexed corpus is written in precise domain terminology. For example, a user may ask about “weird network traffic,” while the relevant knowledge-base entries use terms such as command-and-control, network service discovery, lateral movement, or exfiltration. When this mismatch occurs, retrieval quality drops and the final answer becomes weak or unsupported.

This project therefore studies the following research question: *Does query reformulation using LLM-based agents*

improve retrieval relevance and final answer quality in defensive security RAG systems?

To answer this question, we designed and evaluated a full local cybersecurity RAG system. The system ingests defensive security knowledge, embeds it into a vector database, supports baseline and reformulated retrieval paths, and measures performance using retrieval and grounding metrics across a mixed benchmark of vague and specific security questions.

II. PROJECT GOAL AND SYSTEM OVERVIEW

A. What We Are Trying to Build

The project goal is to build an agentic AI assistant for defensive security. A user asks a cybersecurity question, the system retrieves relevant knowledge from a security corpus, and an LLM generates an answer from those retrieved documents. The experimental contribution is the addition of a query reformulation layer that tries to improve the search step before answer generation.

B. Knowledge Base and Document Store

The knowledge base combines three public cybersecurity sources that are realistic and manageable for a student research project. First, MITRE ATT&CK provides structured adversary techniques, tactics, platforms, and descriptions. Second, the National Vulnerability Database provides CVE records and vulnerability descriptions. Third, the CISA Known Exploited Vulnerabilities catalog provides remediation-oriented entries for vulnerabilities observed in real-world exploitation. These sources were chunked and embedded into a persistent Chroma vector store.

C. End-to-End Workflow

The full workflow has four stages. Stage 1 ingests and chunks the cybersecurity documents. Stage 2 embeds those chunks and stores them for dense retrieval. Stage 3 runs either the baseline or a reformulation strategy to retrieve top-k evidence. Stage 4 prompts the LLM to answer using the retrieved context, after which benchmark metrics are computed.

III. PIPELINE DESIGN

Five pipelines were evaluated in this project. The baseline serves as the control condition, and four enhanced pipelines apply different query-rewriting strategies before retrieval. Table I summarizes their behavior, and the following subsections explain each one clearly.

TABLE I

Pipeline	What It Does
Baseline	Uses the raw user query exactly as typed, retrieves the top security chunks, and generates an answer from those retrieved documents.
Expansion	Rewrites the raw query by adding cybersecurity language such as attack behaviors, MITRE-style terms, and detection-oriented wording.
HyDE	Generates a hypothetical answer passage first, then embeds that passage as the retrieval query so the search resembles the language of the corpus.
Step-Back	Generalizes the question to a broader security concept before retrieval so the model can pull more foundational context.
Decomposition	Breaks a complex query into smaller sub-questions, retrieves evidence for each one separately, then merges the evidence for answer generation.

Pipelines Evaluated in the RAGDMS System

A. Baseline Pipeline

The baseline pipeline is the simplest control system. A user query is embedded directly with nomic-embed-text, the top retrieved documents are loaded from ChromaDB, and Llama 3.2 answers using that evidence. This pipeline is important because it shows how well a normal RAG system performs without any extra intelligence in the query stage.

B. Query Expansion

Query expansion attempts to enrich an underspecified query with security terminology. The purpose is to reduce the lexical gap between human phrasing and security documentation. For example, a vague report about unusual logins or network anomalies can be reframed using concepts such as brute force, credential stuffing, lateral movement, beaconing, or exfiltration. The intended benefit is better recall when the user does not know the correct technical phrase.

C. Hypothetical Document Embedding (HyDE)

HyDE does not merely rewrite the question. Instead, it asks the LLM to generate a short hypothetical answer as though it were already a snippet from a relevant security article. That generated paragraph is then embedded and used for retrieval. The intuition is that a hypothetical article-like passage is often closer to the structure and vocabulary of real indexed documents than a short user query is.

D. Step-Back Prompting

Step-back prompting transforms a narrow query into a broader conceptual question. The system tries to retrieve more general supporting knowledge that may still help answer the original problem. This can be useful when the original question is too narrow, too sparse, or expressed in a way that does not match exact corpus entries.

E. Sub-Query Decomposition

Sub-query decomposition is designed for multi-part questions. Instead of sending one broad request to the retriever, the system breaks the query into smaller answerable units. Each sub-query retrieves its own evidence, and the results are merged. In principle, this should improve coverage for compound questions such as detection-plus-response or root-cause-plus-mitigation scenarios.

IV. EXPERIMENTAL SETUP

A. Implementation Stack

Component	Configuration Used
LLM	Llama 3.2 via Ollama
Embedding model	nomic-embed-text via Ollama
Vector store	ChromaDB persistent store
Framework	LangChain-based project structure with custom ingestion and evaluation modules
Corpus sources	MITRE ATT&CK, NVD CVE data, and CISA KEV advisories
Benchmark queries	15 total: 8 vague queries and 7 specific queries
Retrieval depth	Top-5 documents per query

B. Benchmark Dataset

The benchmark dataset contains 15 cybersecurity questions with manually written ground-truth answers and keyword sets. Eight questions are vague, symptom-style analyst queries such as “something weird in my network traffic” or “admin account doing things at 3am.” Seven are specific technical questions such as “How does Pass-the-Hash work?” and “How does DMARC protect against email spoofing?” This split allows the paper to study not only whether reformulation helps, but also when it helps.

C. Evaluation Metrics

Four metrics were used. Recall@5 measures whether the retriever found relevant material within the top five documents. MRR measures how early the first relevant result appears in the ranking. Answer overlap is a lexical proxy for answer accuracy relative to the prepared ground-truth answer. Faithfulness measures whether the generated answer appears grounded in the retrieved context rather than unsupported model memory.

V. RESULTS

Table II reports the mean scores across all benchmark queries. The strongest overall pipeline is HyDE, which achieved the highest Recall@5 and MRR. The baseline remained competitive in faithfulness and performed well on some already-specific questions. Expansion and step-back prompting were weaker overall, while decomposition helped some vague queries but did not beat HyDE in the aggregate.

TABLE II

Pipeline	Recall@5	MRR	Ans. Overlap	Faithfulness
Baseline	0.270	0.478	0.049	0.400
Expansion	0.178	0.506	0.014	0.128
HyDE	0.383	0.639	0.050	0.386
Step-Back	0.147	0.278	0.015	0.200
Decomposition	0.217	0.394	0.024	0.238

Mean Benchmark Scores Across All 15 Queries

TABLE III

Pipeline	Vague	Specific	Interpretation
Baseline	0.181	0.371	Strong on already-specific queries
Expansion	0.119	0.245	Prompt drift reduced usefulness
HyDE	0.275	0.507	Best overall retrieval in both groups
Step-Back	0.150	0.143	Too broad for many security questions
Decomposition	0.244	0.186	Useful mainly for broader vague questions

Recall@5 by Query Category

A. What the Results Actually Show

The experimental results show that query reformulation is not automatically beneficial. Instead, its value depends on the specific strategy and the type of query. HyDE performed best overall because its hypothetical passage often matched the style of cybersecurity source documents. Query expansion, although conceptually attractive, often suffered from drift and over-specification. Step-back prompting was generally too abstract. Decomposition improved some vague questions but introduced extra complexity without consistent gains.

The baseline pipeline remained important and competitive. For several already-technical questions, the raw query itself was a strong retrieval signal. This confirms a practical lesson for defensive security systems: an intelligent front end should not rewrite every query blindly. A selective routing policy is likely better than a one-size-fits-all reformulation layer.

B. Qualitative Examples from the Demo

The screenshots captured during the local demonstration illustrate why the quantitative results matter. In some vague cases, reformulation produces a clearer retrieval path and a more security-aware explanation. In other cases, especially when the model invents overly specific or incorrect details, reformulation can push retrieval away from the correct evidence.

```

=====
DEMO CASE: Vague Query - "users getting locked out randomly"
=====
BASELINE - Raw Query: "users getting locked out randomly"

Answer:
Based on the provided context, it appears that the issue of users getting locked out randomly could be related to brute force attacks on VNC libraries
implementations. The citations mention specific vulnerabilities such as "hijacking VNC" and "weak root VNC login without authentication", which suggest
that unauthorized access attempts can lead to user lockouts.

It is possible that the issue of users getting locked out randomly is due to an attacker attempting to brute force their way into a VNC session, causing
the account to be locked out after a certain number of failed login attempts. This would require further investigation and analysis to determine the
root cause and implement appropriate mitigations to prevent such attacks.

However, without more specific information about the environment, such as the operating system, version of the VNC library, or any other relevant det
ails, it is difficult to provide a more precise answer.

Recall@5=0.25 MRR=1.00 Faithfulness=0.20

=====
ENHANCED (EXPANSION)
Reformulated: "Are there any instances of brute-force attacks, session hijacking, or credential stuffing occurring on our system, poten"

Answer:
Based on the provided context, users getting locked out randomly could be a sign of brute forcing credentials. Adversaries may attempt to brute force
access to valid accounts within a victim environment, leveraging knowledge gathered from other post-compromise behaviors such as OS Confidential Dumping,
Account Discovery, or Password Policy Discovery.

This behavior is consistent with the tactics, techniques, and procedures (TTPs) described in the context, where adversaries use various techniques to
gather information about valid accounts and then attempt to brute force access to them. Therefore, if users are getting locked out randomly, it could be
a indication of an adversary attempting to brute force credentials within the environment.

Recall@5=0.25 MRR=0.33 Faithfulness=0.50

```

Fig. 1. Demo case for a vague account-lockout query showing baseline and reformulated outputs.

```

=====
DEMO CASE: Specific Query - "How does Pass-the-Hash work and how can I detect it?"
=====
BASELINE - Raw Query: "How does Pass-the-Hash work and how can I detect it?"

Answer:
Pass-the-Hash is a type of attack that exploits the Kerberos authentication protocol to gain unauthorized access to systems. Here's how it works:

1. Initial Attack: An attacker gains access to a user's credentials, either through phishing, social engineering, or other means.
2. Obtaining the Service Ticket: The attacker uses the stolen credentials to obtain a service ticket from the Kerberos authentication server. This
   ticket contains the hash of the user's password and is used to authenticate the user to the system.
3. Passing the Hash: The attacker passes the hash value obtained in step 2 to the target system, which then uses it to authenticate the user. Sin
   ce the hash value is already known, the attacker can use it to gain access to the system without needing to provide the actual password.

To detect Pass-the-Hash attacks:

1. Monitor Kerberos Traffic: Analyze Kerberos traffic on your network to identify suspicious activity. Look for unusual patterns, such as multiple
   service tickets being requested in a short period.
2. Analyze Authentication Logs: Review authentication logs to identify users who are attempting to access systems without providing their passwo
   rd. This could indicate a Pass-the-Hash attack.
3. Deploy Intrusion Detection Systems (IDS): IDS can help detect unusual network activity that may indicate a Pass-the-Hash attack.
4. Implement Additional Security Measures: Consider implementing additional security measures, such as:
   - Using secure password storage and hashing algorithms.
   - Enforcing strong password policies.
   - Limiting access to sensitive systems and resources.
   - Regularly monitoring system logs and network traffic for suspicious activity.

It's essential to note that Pass-the-Hash attacks can be difficult to detect, especially if the attacker is using a legitimate service ticket. Theref
ore, it's crucial to implement robust security measures and regularly monitor system logs and network traffic to detect such attacks.

Recall@5=0.80 MRR=0.80 Faithfulness=0.80

=====
ENHANCED (EXPANSION)
Reformulated: "Can you explain to me how Pass-the-Hash attacks work, specifically the use of compromised credentials to authenticate an"

```

Fig. 2. Demo case illustrating a difficult specific technical query where retrieval remains challenging.

```

=====
SUMMARY - Mean Scores Across All Demo Queries
=====
Pipeline          Recall@5    MRR    Faithfulness
-----
baseline           0.30       0.55    0.46 ← control
enhanced_expansion 0.30       0.38    0.47
enhanced_hyde      0.12       0.25    0.28
enhanced_stepback  0.25       0.33    0.55

=====
KEY FINDING:
Reformulation helps most on vague queries.
HyDE and Query Expansion tend to outperform baseline on recall.
Specific queries sometimes perform equally well without reformulation.

subh00pearsarkar@MBA4-5GFF RAGDMS %

```

Fig. 3. Terminal summary of demo for easier visualisation.

VI. DISCUSSION

A. Why HyDE Worked Best

HyDE worked well because the generated hypothetical answer resembled the narrative style of ATT&CK entries and advisory text. This made the embedding stage more compatible with the indexed corpus. In security retrieval, this is especially useful when the original user wording is informal or symptom-based rather than technique-based.

B. Why Expansion and Step-Back Often Hurt

Expansion and step-back prompting were more fragile. Earlier runs in the project even produced SQL-like reformulations, which is a clear example of model drift. Although those prompts were later corrected, the benchmark still showed that these strategies can distort intent, hallucinate incorrect details, or become too broad to retrieve the most useful evidence.

C. Limitations

The query set contains 15 examples rather than a large industrial testbed. And the answer overlap is only a proxy for full factual correctness. Ground-truth relevance is based on prepared keyword sets rather than expert annotation. Finally, the system uses a local lightweight model through Ollama, so larger models may change the quality of reformulation.

VII. CONCLUSION

This paper presented a complete defensive security RAG prototype and evaluated whether query reformulation improves retrieval quality. The answer is nuanced. Reformulation can help, but only when the strategy matches the query. In this project, HyDE was the best-performing reformulation method overall, while the baseline remained strong for already-specific technical questions. The work therefore supports a practical recommendation: future security assistants should classify query type first, then

selectively apply reformulation rather than forcing it on every request.

Future work can extend this project with a larger labeled benchmark, log-centric datasets, stronger local models, and an adaptive controller that chooses among baseline, HyDE, expansion, or decomposition based on the incoming question.

ACKNOWLEDGMENT

This paper is based on the implementation, benchmarking, and local demonstration carried out in the RAGDMS project workspace. The work used public security knowledge sources and local Ollama-based inference to preserve an offline experimental workflow.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuhn, V. Stoyanov, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.
- [3] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in Proc. ACL, 2023.
- [4] H. Zheng, Z. Feng, Z. Xie, and J. Zhou, “Take a step back: Evoking reasoning via abstraction in large language models,” arXiv:2310.06117, 2023.
- [5] MITRE, “ATT&CK knowledge base,” 2026. [Online]. Available: <https://attack.mitre.org>
- [6] NIST, “National vulnerability database,” 2026. [Online]. Available: <https://nvd.nist.gov>
- [7] CISA, “Known exploited vulnerabilities catalog,” 2026. [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>